

Informe Tarea 1 MongoDB

Nombre: Gabriel Escalona

Rol: 202173610-7

Rut: 20755802-8

El repositorio donde se encuentran los archivos de evidencia y ejecución es:

<https://github.com/GbNok/Tarea1BasesDeDatosAvanzadas>

Preparación:

Lo primero que se realizó fue generar un dataset de 20 elementos con las características mencionadas. Para la creación de este dataset, se utilizó la inteligencia artificial de Gemini. Especificando la estructura base y también que al momento de crear los personajes de cada actor, se creen 1 o más para cada uno. Este dataset se encuentra en el archivo

Gemini-base-dataset.json.

Para poder hacer funcionar los contenedores de docker en mi máquina se tuvieron que realizar unos pequeños cambios al docker-compose como agregar volúmenes para cada servicio.

Al tener los contenedores de docker corriendo y funcionando me puede conectar utilizando MongoDB Compass. Estableciendo la conexión a:

`mongodb://localhost:30001,localhost:30002,localhost:30003/?replicaSet=my-replica-set`

Al estar funcionando todo esto se realizó la creación de la base de datos de Actores y la colección de Voice-actors. En esta colección se subió el archivo con los datos generados por Gemini.

En este momento ya está todo listo para comenzar el desarrollo.

Parte 1 Consultas.

En el repositorio de la tarea se pueden encontrar todos los comandos utilizados para cada sección en el archivo `commands.md`

- Consulta 1:

El comando utilizado fue:

```
db["Voice-actors"].find({Edad: {$gt: 50}}, {_id: 0, Nombre: 1, Edad: 1, }).sort({Edad: 1}).limit(10)
```

El resultado fue

```
> db["Voice-actors"].find({Edad: {$gt: 50}}, {_id: 0, Nombre: 1, Edad: 1, }).sort({Edad: 1}).limit(10)
< {
  Nombre: 'Valeria Castro',
  Edad: 52
}
{
  Nombre: 'Sofía Reyes',
  Edad: 55
}
{
  Nombre: 'Haruto Suzuki',
  Edad: 58
}
{
  Nombre: 'Elena Rossi',
  Edad: 63
}
{
  Nombre: 'Felipe Vargas',
  Edad: 66
}
{
  Nombre: 'Yuki Tanaka',
  Edad: 71
}
{
  Nombre: 'David Brown',
  Edad: 74
}
```

- Consulta 2:

El comando utilizado fue:

```
db["Voice-actors"].find({Idiomas: "Inglés", Nacionalidad: "Reino Unido"}, {_id: 0, Nombre: 1, Idiomas: 1, Nacionalidad: 1 }).sort({Nombre: 1}).limit(10)
```

El resultado fue

```
> db["Voice-actors"].find({Idiomas: "Inglés", Nacionalidad: "Reino Unido"}, {_id: 0, Nombre: 1, Idiomas: 1, Nacionalidad: 1 }).sort({Nombre: 1}).limit(10)
< {
  Nombre: 'David Brown',
  Nacionalidad: 'Reino Unido',
  Idiomas: [
    'Inglés',
    'Francés'
  ]
}
{
  Nombre: 'Liam Gallagher',
  Nacionalidad: 'Reino Unido',
  Idiomas: [
    'Inglés'
  ]
}
```

- Consulta 3:

El comando utilizado fue:

```
db["Voice-actors"].find({Edad: {$lt: 30}, Personajes: { $elemMatch: { Rol: "principal", Generos: "Acción"} }}, {_id: 0, Nombre: 1, "Personajes.Rol": 1, "Personajes.Generos": 1, Edad: 1}).sort({Nombre: 1}).limit(10)
```

El resultado fue

```
{ db["voice-actors"].find((Edad: {$lt: 30}, Personajes: { $elemMatch: { Rol: "principal", Generos: "Acción"} }}, {_id: 0, Nombre: 1, "Personajes.Rol": 1, "Personajes.Generos": 1, Edad: 1}).sort((Nombre: 1)).limit(10)
{
  Nombre: 'Anaïs Delacrète',
  Edad: 22,
  Personajes: [
    {
      Rol: 'principal',
      Generos: [
        'Ciencia Ficción',
        'Acción'
      ]
    },
    {
      Rol: 'secundario',
      Generos: [
        'Ciencia Ficción',
        'Aventura'
      ]
    }
  ]
},
  Nombre: 'Isabella Costa',
  Edad: 27,
  Personajes: [
    {
      Rol: 'principal',
      Generos: [
        'MOBA',
        'Fantasía',
        'Acción'
      ]
    }
  ]
}
}
```

```
{
  Nombre: 'Kenji Nakamura',
  Edad: 28,
  Personajes: [
    {
      Rol: 'principal',
      Generos: [
        'MOBA',
        'Acción',
        'Fantasía'
      ]
    },
    {
      Rol: 'principal',
      Generos: [
        'MOBA',
        'Acción'
      ]
    }
  ]
},
  Nombre: 'Lucia Morales',
  Edad: 28,
  Personajes: [
    {
      Rol: 'principal',
      Generos: [
        'MOBA',
        'Acción'
      ]
    }
  ]
},
  Nombre: 'Mikaela Johansson',
  Edad: 26,
  Personajes: [
    {
      Rol: 'principal',
      Generos: [
        'Acción',
        'Anteacción'
      ]
    },
    {
      Rol: 'secundario',
      Generos: [
        'Acción',
        'Anteacción',
        'Drama'
      ]
    }
  ]
}
}
```

- Consulta 4:

El comando utilizado fue:

```
db["Voice-actors"].aggregate([{$sort: {Patrimonio: -1}}, {$skip: 4}, {$limit:10}, {$project: {Nombre: 1, Patrimonio: 1, Edad: 1}}])
```

El resultado fue

```
> db["Voice-actors"].aggregate([{$sort: {Patrimonio: -1}}, {$skip: 4}, {$limit:10}, {$project: {Nombre: 1, Patrimonio: 1, Edad: 1}}])
{
  "_id": ObjectId("6a9c867cal233f36525e9722"),
  "Nombre": "Valeria Castro",
  "Edad": 52,
  "Patrimonio": 2100000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9731"),
  "Nombre": "Felipe Vargas",
  "Edad": 66,
  "Patrimonio": 1450000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9725"),
  "Nombre": "James Smith",
  "Edad": 43,
  "Patrimonio": 1150000
}
{
  "_id": ObjectId("6a9c867cal233f36525e972f"),
  "Nombre": "Roberto Gómez",
  "Edad": 49,
  "Patrimonio": 980000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9720"),
  "Nombre": "Sofía Reyes",
  "Edad": 55,
  "Patrimonio": 850000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9730"),
  "Nombre": "Samantha Lee",
  "Edad": 33,
  "Patrimonio": 670000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9728"),
  "Nombre": "Carlos Mendoza",
  "Edad": 35,
  "Patrimonio": 450000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9721"),
  "Nombre": "Liam Gallagher",
  "Edad": 27,
  "Patrimonio": 320000
}
{
  "_id": ObjectId("6a9c867cal233f36525e9732"),
  "Nombre": "Isabella Costa",
  "Edad": 27,
  "Patrimonio": 290000
}
{
  "_id": ObjectId("6a9c867cal233f36525e972d"),
  "Nombre": "Javier Torres",
  "Edad": 31,
  "Patrimonio": 275000
}
```

Parte 2 Procesamiento de Consultas.

Para realizar esta sección lo primero que se realizó fue ejecutar los comandos anteriores con explain de la siguiente forma:

- 1) `db["Voice-actors"].find({Edad: {$gt: 50}}, {_id: 0, Nombre: 1, Edad: 1, }).sort({Edad: 1}).limit(10).explain("executionStats")`
- 2) `db["Voice-actors"].find({Idiomas: "Inglés", Nacionalidad: "Reino Unido"}, {_id: 0, Nombre: 1, Idiomas: 1, Nacionalidad: 1 }).sort({Nombre: 1}).limit(10).explain("executionStats")`
- 3) `db["Voice-actors"].find({Edad: {$lt: 30}, Personajes: { $elemMatch: { Rol: "principal", Generos: "Acción"}}}, {_id: 0, Nombre: 1, "Personajes.Rol": 1, "Personajes.Generos": 1, Edad: 1}).sort({Nombre: 1}).limit(10).explain("executionStats")`
- 4) `db["Voice-actors"].explain("executionStats").aggregate([{$sort: {Patrimonio: -1}}, {$skip: 4}, {$limit: 10}, {$project: {Nombre: 1, Patrimonio: 1, Edad: 1}}])`

Luego se realiza la creación de los índices mencionados de la forma:

- `db.getCollection("Voice-actors").createIndex({ Nacionalidad: 1 })`
- `db.getCollection("Voice-actors").createIndex({ Edad: 1 })`

Después de haber creado estos índices se realiza nuevamente a ejecución de los comandos con el explain. Los resultados de ambas ejecuciones se almacenaron en archivos txt llamados q{n}_explain.txt y q{n}_explain_index.txt. Para cada pregunta. Los archivos no seran adjuntados al informe para mantener el orden de este.

Comparación de Planes de Ejecución

Al analizar las ejecuciones antes y después de los índices podemos ver muchas diferencias:

- Lo primero es como antes de aplicar los índices, las cuatro consultas utilizaron COLLSCAN, mientras que al aplicar los índices, se dejó de utilizar eso y se cambió a IXSCAN (excepto en la consulta 4, dado que los índices no afectaron en nada a la ejecución de este comando).
- Podemos ver que la cantidad de documentos examinados (en la variable TotalDocsExamined), para las ejecuciones sin índices es de 20, osea se revisan todos los documentos de forma secuencial. Mientras que al agregar los índices, para todas las ejecuciones (menos la 4 por lo explicado anteriormente) se examina una cantidad muy reducida de documentos (7 en la primera, 2 en la segunda, 8 en la tercera)
- Al realizar la búsqueda antes de agregar los índices, la cantidad de llaves examinadas (TotalKeysExamined) tiene un valor de 0. Mientras que al agregar los índices, la cantidad de llaves examinadas deja de ser cero (para todos los casos menos el 4).
- Para ambos casos (con y sin índice) la cantidad de documentos retornados fue la misma, dado que el tipo de salida de ambas ejecuciones fue la misma
- Los tiempos de ejecución que se pueden ver (executionTimeMillis), no se pueden ver grandes diferencias considerables en las ejecuciones con o sin índices. Esto puede ser

dado que la cantidad de datos con la que se está trabajando no alcanza a mostrar una diferencia considerable.

- Los planes seleccionados corresponden a COLLSCAN para las consultas sin índices y la consulta 4 con índice. Y IXSCAN para las consultas con los índices.

Escalabilidad

Al realizar un aumento exponencial al volumen de la base de datos, pasando de los 20 documentos a los 10 mil millones, el impacto al rendimiento de las consultas y a la infraestructura de la base de datos sería lo siguiente para estos casos:

- Las consultas ejecutadas sin índices: Las consultas que deban utilizar un COLLSCAN, podrían colapsar el sistema. Dado que MongoDB intenta cargar a memoria y leer secuencialmente 10 mil millones de documentos. Esto saturaría la RAM y los ciclos de CPU del servidor, provocando tiempos de respuesta demasiados largos, los cuales pueden sobrepasar los tiempos de consultas límite configurados, causando a que estos sean abortados.
- Las consultas que pueden utilizar los índices creados: Las consultas con IXSCAN mantendrán tiempos de respuestas eficientes. Esto es dado que en lugar de recorrer la colección entera, se realizara la búsqueda en la estructura jerárquica del índice, aislando y extrayendo únicamente el subconjunto de documentos con las condiciones de la búsqueda
- La cantidad de documentos que MongoDB tendría que examinar: Sin índices, la cantidad de documentos examinados crece de forma lineal y directamente proporcional al tamaño de la colección, obligando a escanear los 10 mil millones de registros. Con índices esta búsqueda se reduce dado que la búsqueda ahora es solo entre documentos que cumplen la condición de búsqueda.
- El tiempo y recursos necesarios para procesar las consultas: Consultas sin índices saturarían la CPU y la RAM en el escaneo secuencial, aumentando el tiempo de ejecución. Mientras que las consultas indexadas liberan más a la CPU y la RAM, manteniendo los tiempos de procesamiento muy bajos.
- El almacenamiento adicional requerido por los índices: Índices para miles de millones de datos pueden requerir muchísimo espacio adicional extra en el disco duro. Y también estos índices pueden añadir latencia a operaciones de escritura como las inserciones, eliminaciones, etc. Esto es dado que el árbol de índices se debe calcular de nuevo al escribir en la colección.

Pero en base a lo observado con los explain(), la utilización de los índices se vuelve fundamental para grandes volúmenes de datos. En colecciones pequeñas la diferencia entre COLLSCAN y IXSCAN es de 0 milisegundos, pero a escala masiva la diferencia es más considerable.

Parte 3 Flexibilidad del Esquema, Resiliencia y Actualización.

Flexibilidad del esquema

- ¿MongoDB permitiría insertar el documento?

Sí, permitiría la inserción dado que MongoDB es una base de datos documental con un esquema dinámico. Esto quiere decir que los documentos dentro de una misma colección no necesariamente tienen que tener los mismos campos ni la misma estructura.

- ¿Qué consecuencias podría tener posteriormente?

El problema que puede tener esto a un nivel de consultas. Si se realiza una consulta que ordene o filtre a los actores por fecha de nacimiento, el documento puede ser evaluado como nulo o omitido, alterando un resultado posible. Y también si esto se realiza, una aplicación que utiliza la base de datos y esta espera estrictamente encontrar el campo "Fecha de Nacimiento", esto puede crear un error en la aplicación al no encontrar el dato.

- ¿Considera conveniente permitir documentos con atributos faltantes en este caso?

Para este caso no es conveniente. Dado que el documento exige mantener el campo de "Edad", el cual está ligado a la "Fecha de Nacimiento", por lo que permitir a que este campo falte, puede generar un problema de consistencia de datos, ya que no habrá forma de verificar que la fecha ingresada es correcta o de actualizar la edad de forma automática.

- ¿Existe alguna forma de controlar o validar la estructura de los documentos en MongoDB?

Sí, MongoDB tiene una funcionalidad llamada **Schema Validation**, donde se pueden definir reglas a nivel de colección para obligar a que ciertos campos sean requeridos.

Resiliencia de la base de datos.

Se logó identificar el nodo primario utilizando MongoDB compass, según los resultados que se encuentran en el archivo primary.txt. Podemos ver que el nodo primario corresponde al nodo 3 (mongo3). Utilizando docker se detuvo el servicio y se realizó una query sencilla para comprobar si es que todo sigue funcionando:

```
> db["Voice-actors"].find({}, {Nombre: 1}).limit(2)
< {
  _id: ObjectId('6a9c867ca1233f36525e971f'),
  Nombre: 'Kenji Nakamura'
}
{
  _id: ObjectId('6a9c867ca1233f36525e9720'),
  Nombre: 'Sofía Reyes'
}
```

En esta foto podemos ver el resultado de la ejecución de la query, y podemos ver que sigue funcionando de forma correcta.

La razón por la que esto sigue funcionando de forma correcta es debido a que la base de datos está configurada como un **Replica Set** de 3 nodos, donde los 3 nodos se comunican constantemente mediante a "heartbeats".

Al detener el nodo 3 (mongo3) los nodos secundarios dejaron de recibir heartbeats del nodo primario. Al detectar la caída del nodo primario se realiza una nueva elección de nodo primario, donde uno de los nodos secundarios pasa a ser seleccionado como el nuevo nodo primario.

Mediante a este mecanismo se asegura la alta disponibilidad y la resiliencia del sistema, permitiendo que la BD siga recibiendo y analizando consultas.

Actualización masiva de documentos.

Para esta sección yo cometí un error, utilizando distintos nombres para la serie. Para los actores menores de 40 les agregue lo siguiente:

```
{
  "NombrePersonaje": "Adagio",
  "Produccion": "Shangrila",
  "TipoProduccion": "profesional",
  "Rol": "principal",
  "CantidadApariciones": 3,
  "Generos": [
    "Romance"
  ]
}
```

Y para los mayores (o igual) de 40:

```
{
  "NombrePersonaje": "Nature of Daylight",
  "Produccion": "Paltra",
  "TipoProduccion": "profesional",
  "Rol": "secundario",
  "CantidadApariciones": 1,
  "Generos": [
    "Romance"
  ]
}
```

Para realizar la inserción de estos datos utilicé los siguientes comandos:

Para menores de 40:

```
db["Voice-actors"].updateMany({Edad: {$lt: 40}}, {
  $push: {Personajes: { NombrePersonaje: "Adagio", Produccion: "Shangrila", TipoProduccion:
"profesional", Rol: "principal", CantidadApariciones: 3, Generos: ["Romance"]}},
```

```
$mul: {Patrimonio: 1.32}
},)
```

Para mayores de 40:

```
db["Voice-actors"].updateMany({Edad: {$gte: 40}}, {
  $push: {Personajes: { NombrePersonaje: "Nature of Daylight", Produccion: "Paltra",
    TipoProduccion: "profesional", Rol: "secundario", CantidadApariciones: 1, Generos:
    ["Romance"]}},
  $mul: {Patrimonio: 1.32}
},)
```

En el archivo Actors.Voice-actors.json podemos ver los cambios aplicados a los datos originales.

```
> db["Voice-actors"].updateMany({Edad: {$lt: 40}}, {
  $push: {Personajes: { NombrePersonaje: "Adagio", Produccion: "Shangrila", TipoProduccion: "profesional", Rol: "principal", CantidadApariciones: 3, Generos: ["Romance"]}},
  $mul: {Patrimonio: 1.32}
},)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 11,
  modifiedCount: 11,
  upsertedCount: 0
}
> db["Voice-actors"].updateMany({Edad: {$gte: 40}}, {
  $push: {Personajes: { NombrePersonaje: "Nature of Daylight", Produccion: "Paltra", TipoProduccion: "profesional", Rol: "secundario", CantidadApariciones: 1, Generos: ["Romance"]}},
  $mul: {Patrimonio: 1.32}
},)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9,
  modifiedCount: 9,
  upsertedCount: 0
}
```